設計可擴張的 AI 開發流水線

AI Coding Agent 實戰工作坊 — 進階場 (Claude + Codex + Gemini)

目標： 從「管一間廚房」升級到「管多產線工廠」

John Wu | Apr 2026

今天的旅程

時段	做什麼
觀念講解	Agentic 設計思維 + 治理邊界
Live Demo	示範多 Agent 並行 review 流水線
回顧 Q&A	重點整理 + 課後練習

> 今天的關鍵字：**設計流水線**，不是親自操作每一步

Part 1: 觀念講解

從「開餐廳」到「開發流水線」

中階（上一場）	進階（今天）
餐廳：一間廚房怎麼運作	工廠：多條產線怎麼協調
單一 Agent + 規則	多 Agent 編排 + 專案級 orchestration
Prompt / Context / Harness	Agentic Workflow 設計

> 上次你學會開一間廚房，今天你要設計整座工廠的流水線

三大支柱：進階視角

支柱	中階	進階
Prompt	指令鬆緊度	多 Agent 派工
Context	模組化 CLAUDE.md	Playbook 分層（roles/stacks）
Harness	權限 + 反覆調校	Hooks（tool-call 層硬擋）

> 從「會用」升級到「會設計」— 今天逐一展開這些進階用法

Vibe Coding：什麼時候該、什麼時候不該？

適合 Vibe	該用更有結構的方式
即丟腳本 / 一次性爬蟲	效能關鍵路徑、安全相關 code
獨立小功能、utility	跨多系統整合
探索性 prototype	架構決策（AI 不知道你的取捨）

> Vibe Coding 不是不好 — **判斷何時該放手、何時該拿 spec 收斂**

Spec-Driven Development (SDD)

Vibe 不夠時，先寫 spec 再讓 AI 做

開發方式	你做什麼	AI 做什麼
Vibe	說一句話	猜你要什麼
SDD	**先寫清楚 spec**	**照 spec 精準做**

一份 spec 至少回答三題：
What（要做什麼）、**Constraints**（不能做什麼）、**Done**（怎樣算完成）

> Spec 越清楚，AI 產出越可預測、越可驗收
> 後面的 OpenSpec `proposal.md` 就是在回答這三題

為什麼需要工作流框架？

從 Prompt Engineering → Workflow Engineering

痛點	你一定遇過
上下文漂移	長對話後，AI 逐漸忘記前面的約束，品質退化
需求失真	需求只活在對話裡，缺乏結構化沉澱
團隊難複製	個人 prompt 技巧無法推廣成團隊能力
執行不穩定	大型重構、多檔案聯動，缺乏穩定推進機制

> 核心轉變：從「提示詞怎麼寫」→「**工作流怎麼搭**」

Agentic Workflow — 你設計流水線，AI 跑產線

層次	比喻	做法
傳統	手工作坊	你寫所有 code
AI 輔助	你操作機器	你做事，AI 幫忙
Agentic	**你設計流水線**	**你設計流程，AI 自動執行**

你 = 流水線設計師

> **你** 設計流程（spec）+ 品管驗收（review）
> **AI** 負責每一個工站（plan → implement → test）

多 Agent 架構 — 為什麼要拆？

問題：一個 Agent 做太多事 → 品質下降

```
❌ 一個 Agent，又改前端又改後端又寫測試 + 超長對話
   → Context 爆炸 → AI 忘記前面指令 → 品質崩壞
```

解法：每個子 Agent 專注一件事

```
✅ 主 Agent（生產線總監）
   ├── Agent A：只做前端，載入對應 stack + skills
   ├── Agent B：只做後端，載入對應 stack + skills
   └── Agent C：只做測試，載入對應 skills
```

> 跟微服務一樣：**單一職責、介面明確**

多 Agent 的核心收益

不只是「平行加速」，更是 context 衛生

收益	說明
降低 context 密度	每個 session 只帶必要資訊
單一職責	每個 Agent 只專注一種任務
明確整合點	review 與驗收有清楚邊界

> 多 Agent 首先是 **context hygiene**（上下文乾淨），其次才是速度

什麼時候用多 Agent？

適合	不適合
任務已有 plan，可拆明確子題	還在 brainstorm、問題沒收斂
每個子題有清楚輸入輸出	任務太小，拆解成本 > 收益
有整合與 review 點	子題之間高度共享隱性 context

> 正確順序是：**先 plan，再拆 Agent**

多模型協作：Claude Code 調度其他模型

透過 Plugin，Claude Code 可以調度其他模型

Plugin	調度誰	適合場景
codex-plugin-cc	OpenAI Codex	Code review、第二意見
cc-gemini-plugin	Google Gemini	大型 codebase 全局分析

- 個人感受：Codex 擅長規劃，Claude 擅長實作，Gemini 擅長大範圍分析

> 不同模型有不同強項 — 組合起來比單打獨鬥更穩

多模型分工：做事的和審查的要不同

階段	做事的	Review 的
Plan	Codex	Claude
Coding	Claude	Codex
全局分析	Gemini	Codex / Claude

> **同一個模型不該自己做又自己審** — 換模型 = 換視角

Subagent 怎麼被觸發？

兩種派工方式

方式	怎麼觸發	例子
自動派工	描述任務，主 Claude 自己挑 Agent	「做留言板，分角色並行開發」
手動指名	明確指定派哪個 Agent、用哪個模型	「派 code-reviewer 做 review」

> 每個子 Agent 獨立 **context**，主 Claude 只看到最終回報

Subagent 也能指定模型

不只手動切，派工時就決定用哪個等級

```
請同時派兩個 subagent 平行作業：

A（用 opus）：檢查所有 prompts 是否符合 prompt-best-practices.md 的原則，
B（用 haiku）：把 README.md 的標題層級與條列符號統一成 playbook 其他 README 的風格
```

> 在流水線設計階段就決定每個工站用什麼等級的模型 — 不是事後手動切

Agent Handoff Contract

子 Agent 接什麼、交什麼，要寫清楚

欄位	要定義什麼
責任範圍	只負責哪一段工作
輸入/輸出	接手前拿到什麼、完成後交付什麼
可寫範圍	允許動哪些檔案（例：reviewer 純唯讀）
Done	驗收條件 + 不可做的事

> 沒有 handoff contract，就不是分工，而是把混亂分散出去

設定層級：專案級 vs 個人級

類型	專案級（跟著 repo 走）	個人級（跟著你走）
Prompts	`repo/CLAUDE.md`	`~/.claude/CLAUDE.md`
Settings	`repo/.claude/settings.json`	`~/.claude/settings.json`
Skills	`repo/.claude/skills/`	`~/.claude/skills/`

> 專案級由資深人員維護，個人級自己管理
> 衝突時**專案級優先** — 團隊規範贏過個人偏好

為什麼需要 Playbook？

一個團隊、N 個專案，規範怎麼不失焦

沒有 Playbook	有 Playbook
每個 repo 自寫 CLAUDE.md	共用一套，submodule 引入
新人各專案重讀一次規範	認得一套就懂所有專案
規範更新手動推 N 個 repo	更新 Playbook，各 repo pull 即可

> Playbook = 團隊共用的 AI 工作規範，跨專案一次落地

Playbook — 跨專案共用規範

用 `submodule` 統一管理

- playbook 是 core + overlay：
  `roles / stacks / policies / agents / hooks / skills / templates / guides`
- 所有專案引 submodule → 共用同一套規範

> 不用每個 repo 各寫一份，**一處維護、多處生效**

Playbook 的分層結構

優先層級（高 → 低） + 衝突時誰贏

```
CLAUDE.md                  ← 全域規則（最高）
  → policies/              ← 團隊紅線，例：資安規範、資料分級、PR 必審才能 merge
    → roles/               ← 角色視角
      → stacks/            ← 技術規則
        → skills/          ← 工作方法
          → templates/     ← 輸入輸出模板（最低）
```

> 當 Role 說的跟 Policy 衝突 → Policy 贏
> 當 Skill 說的跟 Stack 限制衝突 → Stack 贏

Permission Mode：你信任 AI 多少？

模式	AI 可以做什麼	適合場景
default	多數操作都要你批准	高風險、不熟悉的任務
acceptEdits	檔案編輯自動過，其他批准	日常開發
bypassPermissions	全部跳過權限詢問	有充分護欄的自動化流程

> 信任程度要跟護欄強度匹配 — **越寬鬆，Hook 越不能少**

Agentic 常見失敗模式

症狀	根因	對策
Agent 偏離 plan	Spec 不夠明確	補強 spec 的 constraints
子 Agent 產出互相衝突	缺少共享的介面約定	先定 API contract 再分工
越做越慢、品質下降	Context 被前面的錯誤污染	開新 session、帶入乾淨檔案

> 大部分 Agentic 失敗不是 AI 不夠聰明，而是 **spec 或 context 出了問題**

失敗隔離與 Rollback — 讓錯誤停在工站

做法	目的
每個子 Agent 獨立 context	記憶不互相汙染（可再搭 git worktree）
有副作用前先停下來等 review	擋住發 MR、動 DB 等不可逆操作
出錯就重開乾淨 session	避免髒 context 繼續累積

> 進階不是避免出錯，而是讓錯誤 **可隔離、可回退、可重來**

Part 2: Live Demo — 多 Agent 流水線

主 Claude 實作 + 多個 reviewer 並行審查

載入 Playbook

初始化專案

```
mkdir live-demo && cd live-demo && git init
git submodule add https://github.com/johnwu1114/playbook.git
./playbook/setup.sh
```

> 一個命令把 **Prompt**、**Context**、**Harness** 三層規範同時落地
> ⚠️ 首次 `claude` 會跳 `@playbook/CLAUDE.md` approval dialog — **務必 approve**（誤拒後 import 會停用，要手動 re-enable）

提需求

下指令給 Claude

```
做一個留言板，包含前端、後端、資料庫用 docker 起。
設計完，分成三個角色並行開發，整合完跑起來給我看。

(不要逐項問我規格，不載入 skill)
```

> 不指定技術棧，依 `@playbook/CLAUDE.md` 挑要載入哪個 stack / policy 覆蓋

並行派 4 個 playbook reviewer

下指令給 Claude

```
並行派 code-reviewer / qa-checker / security-reviewer / migration-reviewer
review 剛剛的產出
```

多模型協作

下指令給 Claude

```
請 codex 做 adversarial-review；
若 codex 建議跨檔影響分析，再派 gemini 支援。
```

Hooks 何時觸發

關鍵：hooks 掛在 Claude 的 tool call

`policies/agent-security.md` 寫進 system prompt 讓 AI 自律（soft）— demo 時 AI 根本不會下這些指令。
Hook 是最後一道硬防線：AI 真的嘗試執行時才擋（hard）。

```
cat > .env <<< "OPENAI_KEY=sk-sample"   # secret-guard 擋
rm -rf ~/sample                         # dangerous-bash 擋
```

> Policy（prompt 層，靠 AI 自律）+ Hook（tool-call 層，硬擋）
> **兩層都在，才叫 defense in depth**

Demo 回顧 — 每一步對應的支柱

Demo 步驟	做了什麼	對應支柱
載入 Playbook	一行 setup 把 roles/stacks/policies 引入	**Context**
提需求	三角色並行實作（前端 / 後端 / DB）	**Prompt**
並行 review	4 reviewer 並行，不同視角審查	**Prompt**
多模型協作	Codex 做 adversarial，條件式升級 Gemini	**跨模型**
Hooks 何時觸發	policy（prompt 層）+ hook（tool-call 層）	**Harness**

> **主 Claude 沒有單打獨鬥**
> Playbook 帶知識、Agent 分工實作、reviewer 換視角、Hook 守底線

Part 3: 課後練習

把今天的方法套到你的工作場景

課後練習 — 打造你的 review 流水線

1. **在真實 MR 上試跑**
   `code-reviewer` + `qa-checker` + `security-reviewer`
2. **觀察報告差異**
   誰提出「交叉驗證建議」？有沒有互相覆蓋的維度？
3. **新增一個屬於你的 Agent**
   例：`i18n-reviewer` / `perf-reviewer` / `api-contract-reviewer`
4. **把它放進團隊 Playbook**
   好的 Agent 是團隊資產，不是個人偏好

> 流水線的價值在**長期累積**：每多一站，整條線就多一層把關

兩週落地路徑

第 1 週	第 2 週
用內建 reviewer 跑 MR	新增一個自訂 reviewer
觀察哪些建議重複	放進團隊 playbook，邀人試跑
筆記常見誤報	收 feedback，調 prompt

> 不要一次改整條流水線 — **先做一站，驗證再擴**

Part 4: 回顧

怎麼知道流水線有變好？ — Agentic 的觀測指標

指標	它在反映什麼
一次通過率	低了通常代表 spec 或 contract 不清楚
Review 打回率	高了代表交付品質不穩或驗收點太晚
人工介入次數	高了代表自動化邊界還不可信
Lead time	變長代表 handoff 太多或整合卡住
Token / task	上升通常代表 context 污染或拆太細；**多 Agent + 多模型時尤其需要關注**
Rollback 次數	高了代表風險控制設計得太後面

多 Agent 流水線的收益與成本

面向	說明
Context hygiene	每個 reviewer 獨立 context — 不被實作對話污染
品質多維度	code / qa / security 各自專業視角，不會互相覆蓋
並行省時	三站同時跑，比排隊快 2–3 倍
團隊資產累積	Agent `.md` 檔能共用、版本化、跨專案重用
維護成本	Agent prompt 要隨團隊規範演化，不能一次寫完擺著
風險邊界	reviewer 是建議者，最後決策仍在主 Claude / 人

> 流水線的價值在長期累積，但你還是要設計人為關卡與最終審核點

關鍵 Takeaway

1. 寫的跟審的永遠不同一個

主 Agent 實作、reviewer Agents 審查；獨立 context 避免自我偏見

2. 每個工站的 prompt / model / 工具都為它的職責設計

sonnet 審邏輯、haiku 做對照題、**Gemini 吃 1M context 做跨檔影響分析**；reviewer 只給唯讀工具

3. 流水線的延伸性來自檔案化的 Agent

一個 `.md` = 一個新工站；團隊 playbook 讓它們變成共享資產

> 進階不是更會下 prompt — 是會設計讓 AI 之間**互相把關**的流水線

三場工作坊回顧

場次	你學會的
初階	體驗 AI 協作，完成第一次可驗收的協作
中階	設計 AI 工作環境，讓結果可重複
進階	設計開發流水線，讓 AI 替你跑流程

> 從「我用 AI」→「AI 穩定工作」→「AI 替我跑流程」

Q&A